Lab Assignment 1

1. Write a python program to create a neuron and predict its output using the threshold activation function.

```
import numpy as np

BIAS = 1
THRESHOLD = 5

class Neuron:
    def __init__(self,weights,bias,threshold=0):
        '''Initializing neuron with weights, bias, threshold(default is 0)'''
        # Changed the Python List to Array Easier to do calculation(dot prod) in Array
        self.weights = np.array(weights)
        self.bias = bias
        self.threshold = threshold


    def predict(self,inputs):
        '''Predicts if neuron will fire or not, depending on calculated weigthed sum and threshold function'''
        # Weighted sum = v in the slides
        # NumPy Library dot products(scalar product) of 1D array
        weighted_sum = np.dot(self.weights,inputs) + self.bias
        if weighted_sum > self.threshold:
            return 1
        return 0

# Prompting Input from the user
no_of_inputs = int(input("Enter the number of inputs of neuron: "))

# List of input(x)
inputs=[]
# List of Weight(w)
weights =[]

# Values of input(x) and weigths(w)
for i in range(0,no_of_inputs):
    x=float(input(f"Enter the value of Input x{i+1}: "))
    w=float(input(f"Enter the value of Weight w{i+1}: "))
    inputs.append(x)
    weights.append(w)

# Creating a object
neuron = Neuron(weights=weights,bias=BIAS,threshold=THRESHOLD)

# Final output of the ANN (y)
output = neuron.predict(inputs=inputs)
```

```
if output ==1:
    print(f"Neuron Fired. y={output}")
else:
    print(f"Neuron not Fired. y={output}")
```

Output

```
Enter the number of inputs of neuron: 1
Enter the value of Input x1: 1
Enter the value of Weight w1: 1
Neuron not Fired. y=0
```

2. Write a python program to train AND Gate Using Perceptron Learning Algorithm.

```
import numpy as np

EPOCHS = 10
LEARNING_RATE = 1

# Step function (Activation function)
def step_function(value):

    # Threshold at 0
    return 1 if value >= 0 else 0

class Perceptron:

    def __init__(self, input_size, learning_rate=1):
        # Initialize weights as 0 (Can be Initialized Randomly as well)
        self.weights = np.random.rand(input_size)  # Makes NumPy Array of <input_size>
        # Initialize bias as 0 (Can be Initialized Randomly as well)
        self.bias = np.random.rand()
        self.learning_rate = learning_rate

    def predict(self, inputs):
        """Takes Inputs array to predict the output"""
        weighted_sum = np.dot(inputs, self.weights) + self.bias

        # Returning after calling the Activation Function
        return step_function(weighted_sum)

    def train(self, training_inputs, target_outputs, epochs=10):
        """Takes the AND Gate (x1,x2) and y to train """
        for epoch in range(epochs):  # Repeat for multiple epochs

            # print(f"\nEpoch {epoch+1}")
            # print(" ----------------------")

            for inputs, target_output in zip(training_inputs, target_outputs):
                prediction = self.predict(inputs)
                error = target_output - prediction  # Calculate (Target Output (t) -  Acutual Output(y))

                # print(f"Input: {inputs}, Prediction: {prediction}, Expected: {target_output}, Error: {error}")

                # Update weights and bias using perceptron learning rule
                self.weights += self.learning_rate * error * np.array(inputs)
                self.bias += self.learning_rate * error

                # print(f"Updated Weights: {self.weights}, Updated Bias: {self.bias}")

# From AND Gate Table (x1,x2)
training_inputs = np.array([
```

```
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
# From AND Gate Table (y) Outputs
target_outputs = np.array([0, 0, 0, 1])

# Initialize and train perceptron
perceptron = Perceptron(input_size=2, learning_rate=LEARNING_RATE)
perceptron.train(training_inputs=training_inputs, target_outputs=target_outputs, epochs=EPOCHS)

# Testing the trained perceptron
print("\nTesting AND Gate Perceptron:")
for inputs in training_inputs:
    print(f"Input: {inputs}, Predicted Output: {perceptron.predict(inputs)}")
```

Output

```
Testing AND Gate Perceptron:
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 0
Input: [1 0], Predicted Output: 0
Input: [1 1], Predicted Output: 1
```

3. Write a python program to implement Min-Max Scalar.

```
import numpy as np

class MinMaxScaler:
    def __init__(self):
        self.min = None
        self.max = None
    def fit(self,data):
        '''Getting the required parameters(min,max)'''
        # axis=0:find the minimum value in each column.
        self.min = np.min(data, axis=0)
        self.max = np.max(data, axis=0)

    # Transform data in range(0,1)
    def transform(self,data):
        '''Normalizing the data in range 0 to 1'''
        # Output Data array
        output = np.round((data - self.min) / (self.max - self.min),3)
        return output

    def fit_transform(self, data):
        '''Combining the fit and transform'''
        self.fit(data)
        return self.transform(data)

# Sample dataset
data = np.array([[5.9, 75],[5.8, 86],[5.2, 50],[5.4, 55],[6.1, 85],[5.5, 62]])

# Create a MinMaxScaler object
scaler = MinMaxScaler()

# Fit and transform the data
scaled_data = scaler.fit_transform(data)
print("Original Data:\n", data)
print("\nScaled Data:\n", scaled_data)
```

Output

```
Original Data:
 [[ 5.9 75. ]
 [ 5.8 86. ]
 [ 5.2 50. ]
 [ 5.4 55. ]
 [ 6.1 85. ]
 [ 5.5 62. ]]

Scaled Data:
 [[0.778 0.694]
 [0.667 1.   ]
 [0.    0.   ]
 [0.222 0.139]
 [1.    0.972]
 [0.333 0.333]]
```

4. Write a python program to implement Standard Scalar.

```
import numpy as np

class StandardScaler:
    def __init__(self):
        self.mean = None
        self.std = None

    def fit(self,data):
        '''Gets the required parameter(mean and SD)'''
        self.mean = np.mean(data, axis=0)
        self.std = np.std(data, axis=0)

    def transform(self,data):
        '''Transforms the data (Normalizes)'''
        return np.round((data - self.mean) / self.std, 3)

    # Applying standard scaling
    def fit_transform(self, data):
        '''Combining fit and transfrom'''
        # calling fit
        self.fit(data)

        #Returning the normalized data
        return self.transform(data)

# Sample dataset
data = np.array([[5.9, 75], [5.8, 86], [5.2, 50], [5.4, 55], [6.1, 85], [5.5, 62]])

# Create a StandardScaler object
scaler = StandardScaler()

# Fit and transform the data
scaled_data = scaler.fit_transform(data)

print("Original Data:\n", data)
print("\nStandardized Data:\n", scaled_data)
```

Output

```
Original Data:
 [[ 5.9 75. ]
 [ 5.8 86. ]
 [ 5.2 50. ]
 [ 5.4 55. ]
 [ 6.1 85. ]
 [ 5.5 62. ]]

Standardized Data:
 [[ 0.808  0.438]
 [ 0.485  1.221]
 [-1.454 -1.339]
 [-0.808 -0.984]
 [ 1.454  1.149]
 [-0.485 -0.486]]
```

5. Write a python program to train perceptron using given training set and predict class for the input (6,82) and (5.3,52)

$Height(x_1)$	$Weight(x_2)$	$Class(t)$
5.9	75	Male
5.8	86	Male
5.2	50	Female
5.4	55	Female
6.1	85	Male
5.5	62	Female

```
import numpy as np

class Perceptron:
    def __init__(self, input_size, learning_rate=1, epochs=5):
        self.weights = np.zeros(input_size)  # Initialize weights to zeros
        self.bias = 0                        # Initialize bias to 0
        self.alpha = learning_rate           # Learning rate
        self.epochs = epochs                 # Number of epochs
        self.X_min = None
        self.X_max = None

    def activation_func(self, x):
        """Activation function: Returns 1 for positive, -1 for negative"""
        return 1 if x >= 0 else -1

    def maxmin_normalize(self, X):
        """Normalizing the data between 0 and 1 using Min-Max Scaling with zero-division handling"""

        if self.X_min is None or self.X_max is None:
            self.X_min = np.min(X, axis=0)
            self.X_max = np.max(X, axis=0)

        # PROBLEM: Division by zero if max and min same
        denominator = self.X_max - self.X_min

        #SOLUTION: Replacing the 0 with 1 to avoid division by zero
        denominator[denominator == 0] = 1

        return np.round((X - self.X_min) / denominator, 3)

    def train(self, X, y):
        """Trains the perceptron using given inputs and target values"""
        # Normalize data before training
        X = self.maxmin_normalize(X)
```

```
        for epoch in range(self.epochs):
            print(f"Epoch {epoch+1}:")
            for i in range(len(X)):
                # Weighted sum
                v = np.dot(X[i], self.weights) + self.bias

                # Apply activation function
                y_pred = self.activation_func(v)

                # Update weights and bias only if there's a misclassification
                if y_pred != y[i]:
                    weight_update = self.alpha * (y[i] - y_pred) * X[i]
                    bias_update = self.alpha * (y[i] - y_pred)
                    self.weights += weight_update
                    self.bias += bias_update
                print(f"  Sample {i+1}: Weights = {self.weights}, Bias = {self.bias}")

    def predict(self, X):
        """Predicts the class for given input"""
        # PROBLEM: Due to not saving the min and max value of training data scaling problem occured
and correct prediction was not made
        # SOLUTION: Normalize test data using the same scaling as training data
        X = self.maxmin_normalize(X)
        v = np.dot(X, self.weights) + self.bias
        return "Male" if self.activation_func(v) == 1 else "Female"


# Question Dataset
training_data = np.array([
    [5.9, 75], [5.8, 86], [5.2, 50],
    [5.4, 55], [6.1, 85], [5.5, 62]
])
target_outputs = np.array([1, 1, -1, -1, 1, -1])  # 1 = Male, -1 = Female

# Create and train perceptron
perceptron = Perceptron(input_size=2, learning_rate=1, epochs=5)
perceptron.train(training_data, target_outputs)

# Testing on new inputs
test_data = np.array([[6, 82], [5.3, 52]])

for i, test in enumerate(test_data):
    # Passing to the Perceptron Obj to predict their Gender
    prediction = perceptron.predict(np.array([test]))
    print(f"Test case {i+1}: {test} → Predicted class: {prediction}")
```

Output

```
Epoch 1:
  Sample 1: Weights = [0. 0.], Bias = 0
  Sample 2: Weights = [0. 0.], Bias = 0
  Sample 3: Weights = [0. 0.], Bias = -2
  Sample 4: Weights = [0. 0.], Bias = -2
  Sample 5: Weights = [2.    1.944], Bias = 0
  Sample 6: Weights = [1.334 1.278], Bias = -2
Epoch 2:
  Sample 1: Weights = [2.89  2.666], Bias = 0
  Sample 2: Weights = [2.89  2.666], Bias = 0
  Sample 3: Weights = [2.89  2.666], Bias = -2
  Sample 4: Weights = [2.89  2.666], Bias = -2
  Sample 5: Weights = [2.89  2.666], Bias = -2
  Sample 6: Weights = [2.89  2.666], Bias = -2
Epoch 3:
  Sample 1: Weights = [2.89  2.666], Bias = -2
  Sample 2: Weights = [2.89  2.666], Bias = -2
  Sample 3: Weights = [2.89  2.666], Bias = -2
  Sample 4: Weights = [2.89  2.666], Bias = -2
  Sample 5: Weights = [2.89  2.666], Bias = -2
  Sample 6: Weights = [2.89  2.666], Bias = -2
```

```
Epoch 4:
  Sample 1: Weights = [2.89  2.666], Bias = -2
  Sample 2: Weights = [2.89  2.666], Bias = -2
  Sample 3: Weights = [2.89  2.666], Bias = -2
  Sample 4: Weights = [2.89  2.666], Bias = -2
  Sample 5: Weights = [2.89  2.666], Bias = -2
  Sample 6: Weights = [2.89  2.666], Bias = -2
Epoch 5:
  Sample 1: Weights = [2.89  2.666], Bias = -2
  Sample 2: Weights = [2.89  2.666], Bias = -2
  Sample 3: Weights = [2.89  2.666], Bias = -2
  Sample 4: Weights = [2.89  2.666], Bias = -2
  Sample 5: Weights = [2.89  2.666], Bias = -2
  Sample 6: Weights = [2.89  2.666], Bias = -2
Test case 1: [ 6. 82.] → Predicted class: Male
Test case 2: [ 5.3 52. ] → Predicted class: Female
```